COLLABORATIVE SOFTWARE DEVELOPMENT PROJECT

1. PROJECT IDENTIFICATION

1.1 Project Title

AidConnect: NGO Aid Distribution Coordination System

Despite the proliferation of humanitarian and development efforts in dynamic developing regions, such as East Africa (specifically highlighting Kenya), the critical process of aid distribution remains severely plagued by a systemic and persistent **"Coordination Gap."** This deeply entrenched failure significantly diminishes the intended positive impact and overall effectiveness of vast donor resources and the tireless work of countless NGOs. This ongoing crisis in aid delivery is not merely an inconvenience; it is a complex problem defined by three core, interconnected systemic failures:

1. The Invisible Beneficiary (Aid Duplication)

Without a shared cross-organizational ledger, "Aid Duplication" occurs when multiple NGOs unknowingly provide identical resources (e.g., food or medical supplies) to the same household. This "double-dipping" creates an artificial scarcity, where 20% of the population may receive 80% of the available aid, leaving the most vulnerable underserved.

2. Operational Silos (Poor Coordination)

NGOs currently operate as "Data Islands," utilizing independent spreadsheets or internal databases that do not communicate with one another. This lack of shared visibility leads to geographic clustering, where NGOs crowd into accessible urban areas while remote "Aid Deserts" are neglected due to a lack of synchronized logistical planning.

3. The Transparency Deficit (Lack of Accountability)

There is currently no centralized, real-time system to track the lifecycle of an aid item from allocation to physical delivery. This absence of an immutable audit trail reduces donor confidence

and makes it nearly impossible to quantify the collective success of regional humanitarian interventions.

AidConnect is a centralized, web-based coordination platform designed to synchronize humanitarian efforts and optimize resource distribution across diverse non-governmental organizations. By establishing a unified interface for multi-tenant data sharing and logistical tracking, the platform transforms fragmented, siloed operations into a cohesive, data-driven humanitarian network.

1.2 Proposed Solution

The solution is built upon three strategic pillars designed to address the core challenges of aid distribution:

1. Unified Beneficiary Registry and Identity Management

The system implements a secure, cross-organizational registry that utilizes unique identifiers to establish a "Single Source of Truth" for beneficiary data. This ensures that every recipient is uniquely recognized across the entire platform, providing the foundation for accurate need assessments and longitudinal tracking of support.

2. Real-Time Duplication Prevention and Intelligent Prioritization

AidConnect utilizes a predictive logic engine to cross-reference new aid requests against a global, multi-NGO historical ledger. By automatically flagging potential overlaps in aid categories within specific timeframes, the system prevents resource duplication before it occurs. Additionally, the system employs automated prioritization algorithms to rank requests based on urgency and vulnerability levels, ensuring limited resources are directed to those in the most critical need first.

3. End-to-End Logistical Transparency and Impact Analytics

The platform provides a comprehensive tracking mechanism for the entire lifecycle of an aid request, from initial submission through to final physical delivery. Field officers provide real-time status updates via mobile-responsive interfaces, and beneficiaries are integrated into the verification workflow to confirm the successful receipt of aid. This granular data collection generates automated transparency reports and interactive visual dashboards, providing stakeholders with an immutable audit trail of humanitarian impact.

2. TOOLS AND TECHNOLOGIES

The AidConnect technical stack is selected to support a modular, scalable architecture that facilitates collaborative development.

2.1 Frontend Development

- **React.js:** Chosen for its component-based architecture, which allows team members to work on UI elements (e.g., NGO dashboards, beneficiary forms) in isolation without merge conflicts.
- **Tailwind CSS:** A utility-first CSS framework used to ensure a fully responsive interface, meeting the non-functional requirement for mobile-friendly field operations.
- **Chart.js:** Utilized to fulfill the requirement for data visualization and transparency reporting, providing NGOs with interactive insights into aid distribution trends.

2.2 Backend & Database

- **Node.js & Express.js:** Provides a lightweight, high-performance runtime for building the RESTful APIs necessary to handle concurrent user requests and real-time duplication checks.
- **PostgreSQL:** A robust relational database management system (RDBMS) selected for its ACID compliance and support for complex queries, which are essential for maintaining the integrity of the shared beneficiary ledger.

2.3 Version Control & DevOps

- **Git & GitHub:** Serving as the foundation for collaboration, GitHub will be used for version control, peer code reviews through Pull Requests, and issue tracking to manage the 9-week sprint.
- **GitHub Actions:** Implemented to automate the Continuous Integration (CI) pipeline, ensuring that all code is automatically tested for security and functional regressions before deployment.
- **Deployment (Vercel, Render, Railway):** These platforms provide automated deployment environments, allowing for high availability and easy staging of frontend and backend services.

2.4 Project Management & Issue Tracking

- **GitHub Projects:** We will utilize GitHub Projects to implement an **Agile/Kanban** workflow. This allows us to visualize our 9-week roadmap through customized boards, linking every task (Issue) directly to its corresponding code changes (Pull Requests).
- **Issue Tracking:** Every functional requirement will be broken down into granular "GitHub Issues." This ensures that each of the five team members has clear ownership of specific features, with labels used to denote priority levels (e.g., High, Critical) and status (e.g., In Progress, Needs Review).

2.5 Communication & Team Coordination

- **Discord:** To facilitate the "Silicon Savannah" collaborative spirit, Discord will be used for daily "Sync-ups" and "Pair Programming." Its superior low-latency voice and screen-sharing capabilities make it ideal for quick technical huddles and collaborative debugging sessions.
- **Google Workspace:** Used for the collaborative refinement of system requirements, UML models, and the final technical report, allowing for real-time peer feedback through "Suggesting Mode".

3. FUNCTIONAL REQUIREMENTS

These requirements define the specific behaviors and services the system must provide to its users.

3.1 User Authentication and Authorization

- **Secure Identity Management**: The system shall provide a secure registration and login interface for all users, utilizing JSON Web Tokens (JWT) for session management.
- **Role-Based Access Control (RBAC)**: The system shall enforce strict access permissions based on four primary roles: System Admin, NGO Admin, Field Officer, and Beneficiary.

3.2 NGO and Beneficiary Management

- **NGO Verification Workflow**: The system shall allow NGOs to register their credentials, which must be verified by a System Administrator before the organization can manage aid.
- **Unique Beneficiary Identification**: The system shall support the registration of beneficiaries using unique identifiers (such as National ID or system-generated IDs) to ensure a "Single Source of Truth".

3.3 Aid Operations and Logic

- **Automated Priority Scoring**: The system shall automatically calculate a priority score for aid requests based on urgency levels and beneficiary vulnerability metrics.
- **Duplication Prevention Engine**: The system shall cross-reference new aid allocations against a global ledger to prevent a single beneficiary from receiving duplicate aid categories within a defined timeframe.
- **Inventory Resource Allocation**: The system shall facilitate the mapping of approved aid requests to available NGO resources and inventory.

3.4 Delivery and Reporting

- **Real-Time Status Updates**: Field officers shall have the ability to update the status of an aid delivery (e.g., "Pending," "In-Transit," "Delivered") via a mobile-responsive interface.
- **Beneficiary Receipt Confirmation**: The system shall provide a mechanism for beneficiaries to confirm the successful receipt of aid, closing the accountability loop.
- **Analytical Reporting**: The system shall generate and export transparency reports (PDF/CSV) summarizing aid distribution metrics and organizational impact.

4. NON-FUNCTIONAL REQUIREMENTS

These requirements define the system's quality attributes and operational constraints.

4.1 Performance and Usability

- **Latency and Response Time**: All system pages and API endpoints shall load or respond within 3 seconds under normal network conditions.

- **Responsive Interface**: The system's user interface shall be fully responsive, ensuring seamless operation across desktops, tablets, and smartphones.
- **User-Centric Design**: The system shall be designed with high usability standards to minimize the learning curve for field officers with varying levels of technical literacy.

4.2 Security and Reliability

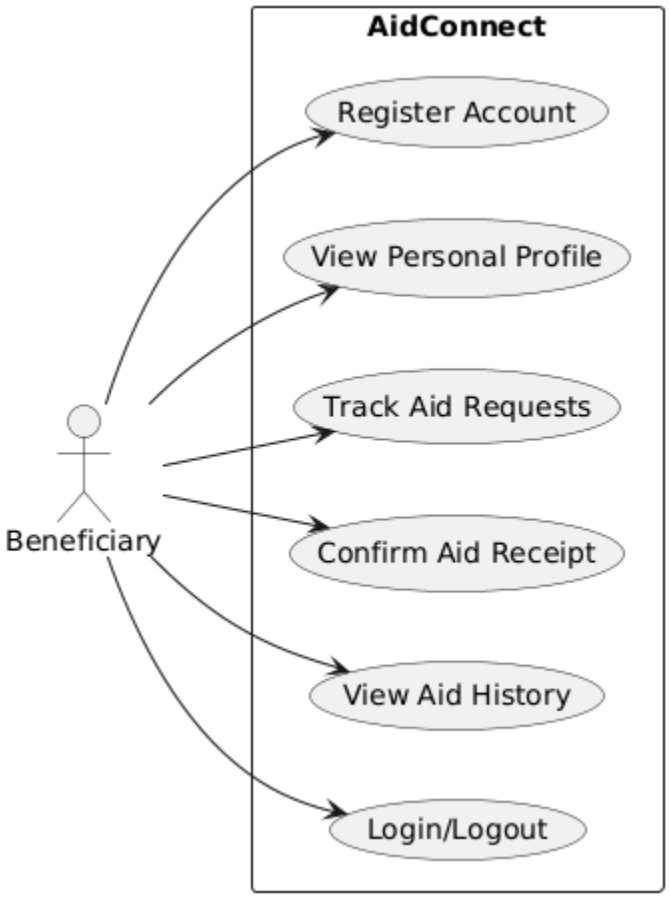
- **Encrypted Communication**: All data transmitted between the client and server shall be secured using Industry-standard HTTPS/TLS encryption.
- **Data Protection**: User passwords and sensitive personal information (like National IDs) shall be encrypted using robust hashing algorithms (e.g., bcrypt) before storage.
- **High Availability and Backups**: The system shall maintain high availability (99.9% uptime) and perform automated daily database backups to prevent data loss.

4.3 Scalability and Maintenance

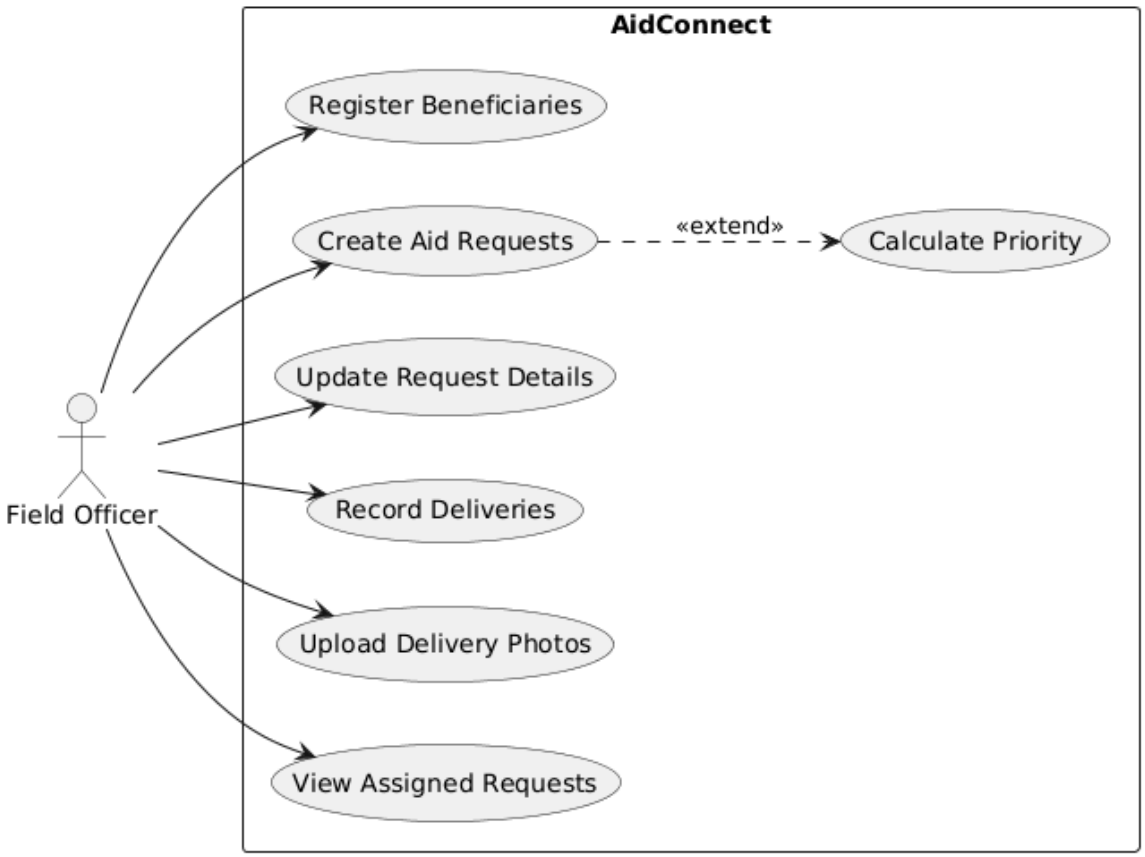
- **Concurrent User Support**: The system architecture shall support multiple concurrent users across different NGOs without degradation in performance.
- **Vertical and Horizontal Scalability**: The system shall be designed to handle an increasing volume of beneficiaries and NGOs by scaling its underlying infrastructure.
- **Code Maintainability**: The project shall follow clean-code principles and modular design to ensure the system remains easy to update and debug throughout the 9-week period.

5. UML Diagrams

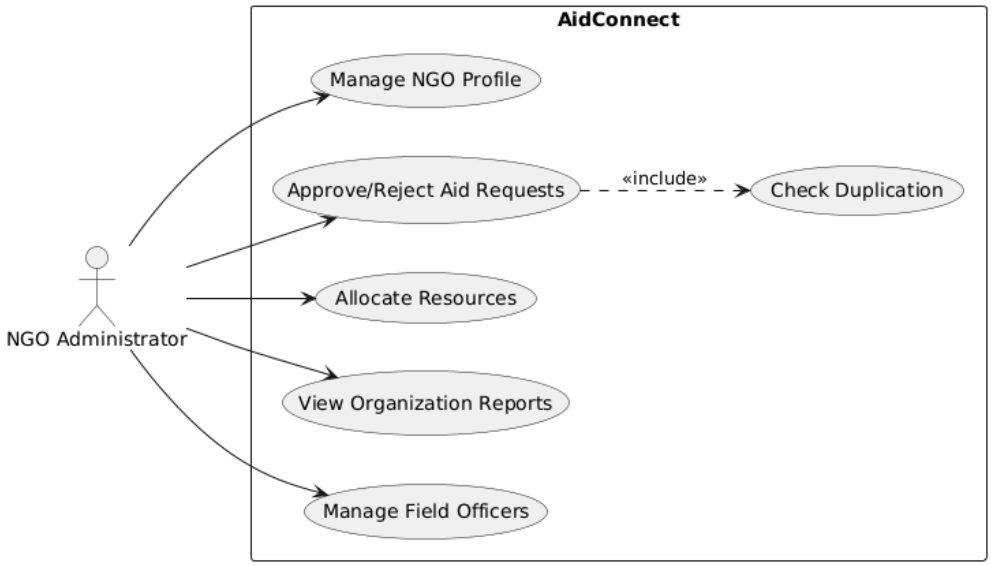
5.1 Use Case Diagrams



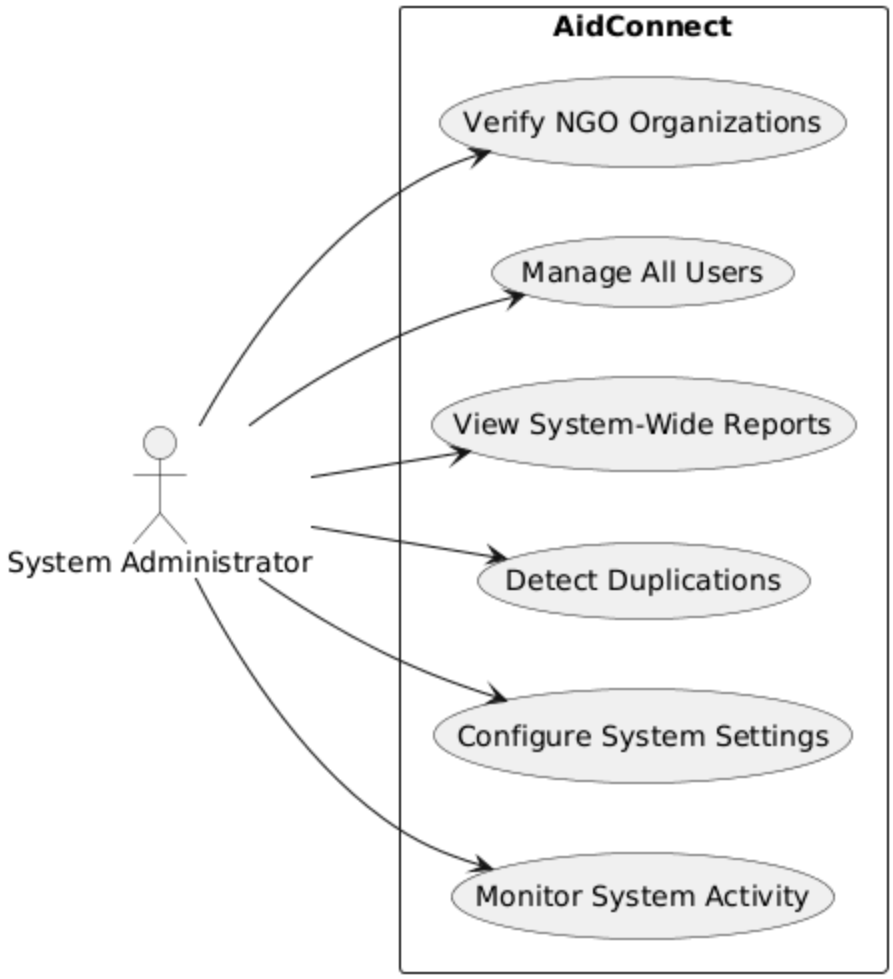
Beneficiary Use Case Diagram



Field Officer Use Case Diagram

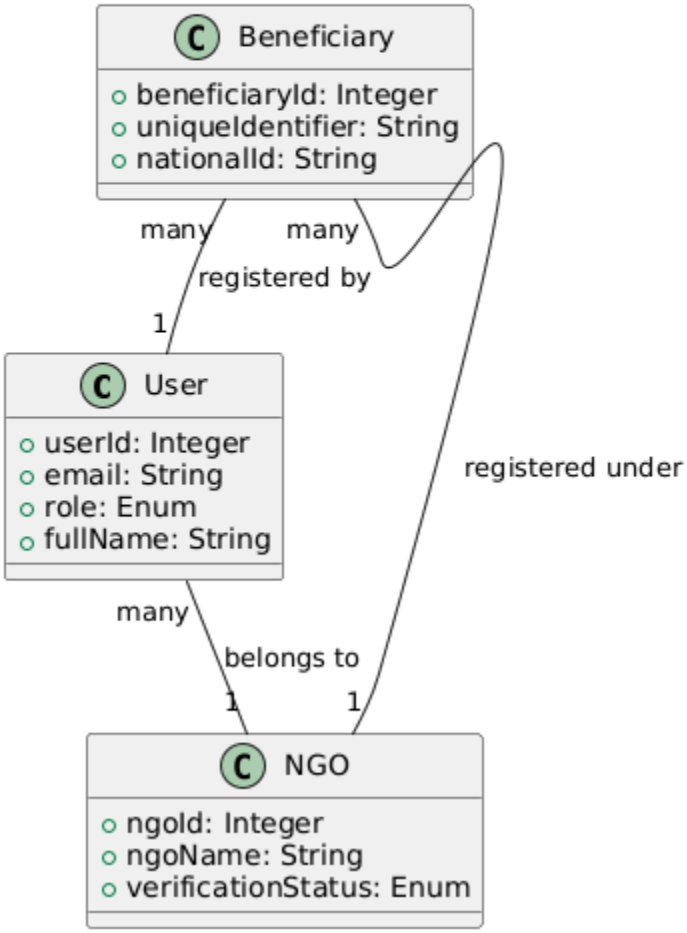


NGO Administrator Use Case Diagram

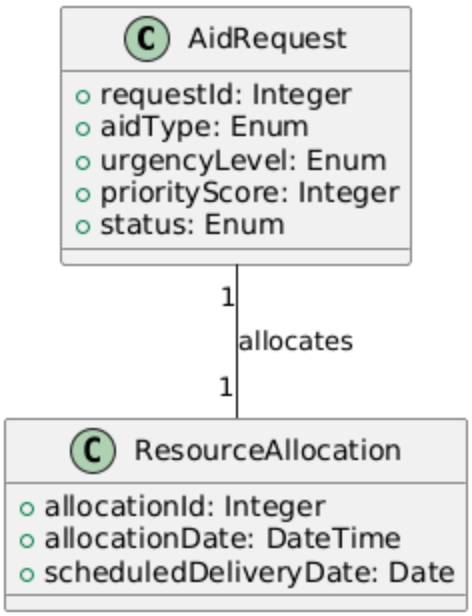


System Administrator Use Case Diagram

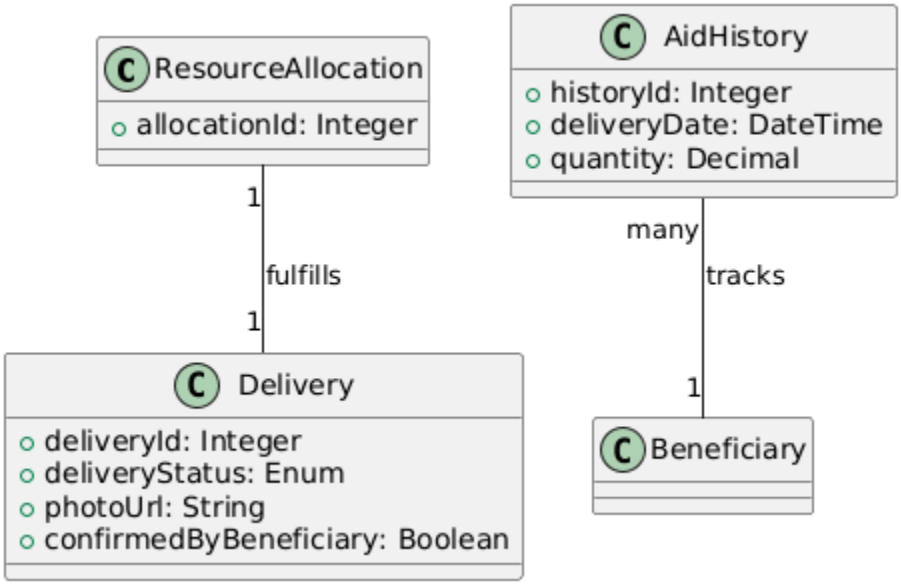
5.2 Class Diagrams



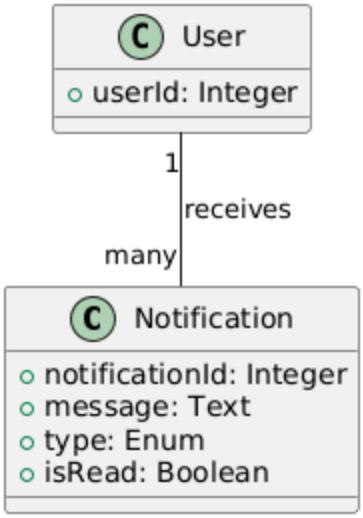
Core Entity Relationship Overview



Aid Request and Allocation Logic

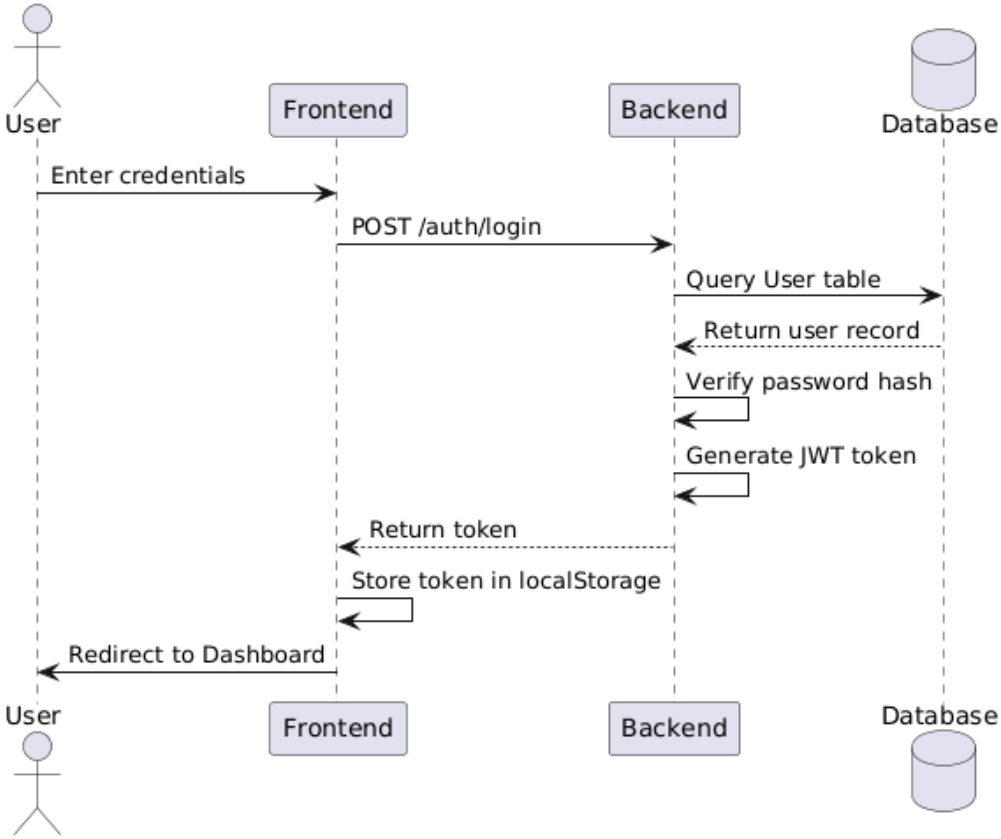


Delivery and History Tracking

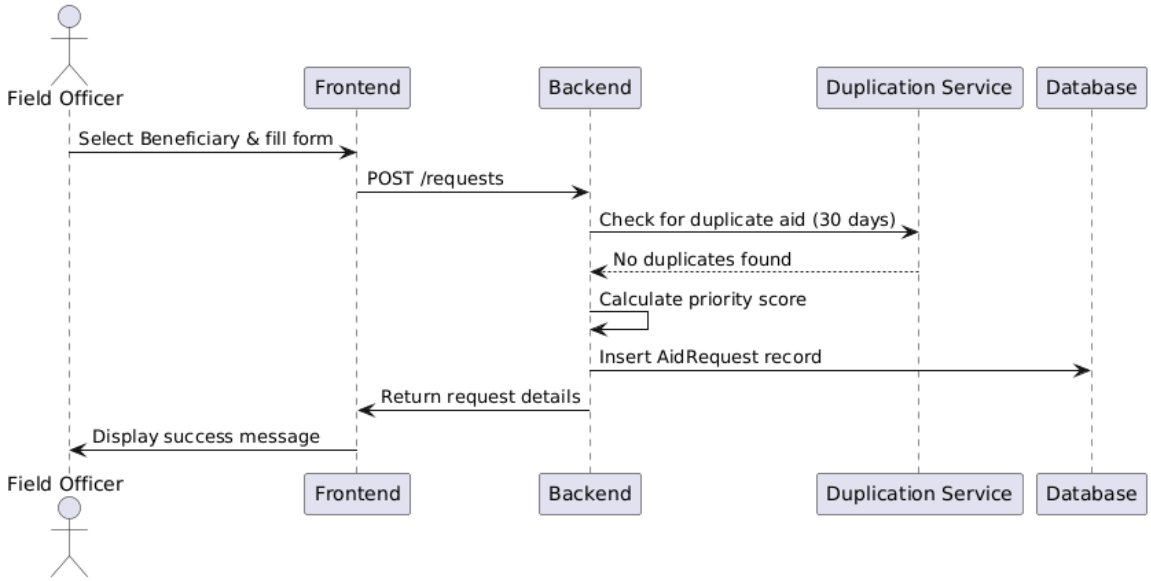


System Communications and Metadata

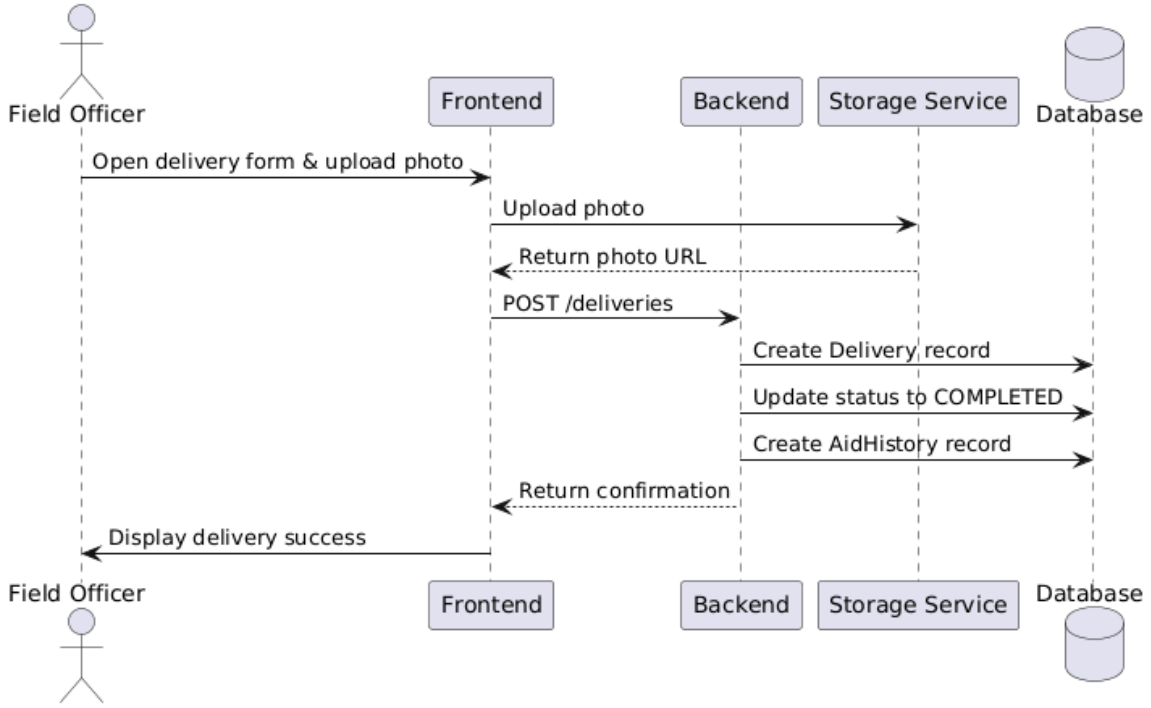
5.3 Sequence Diagrams



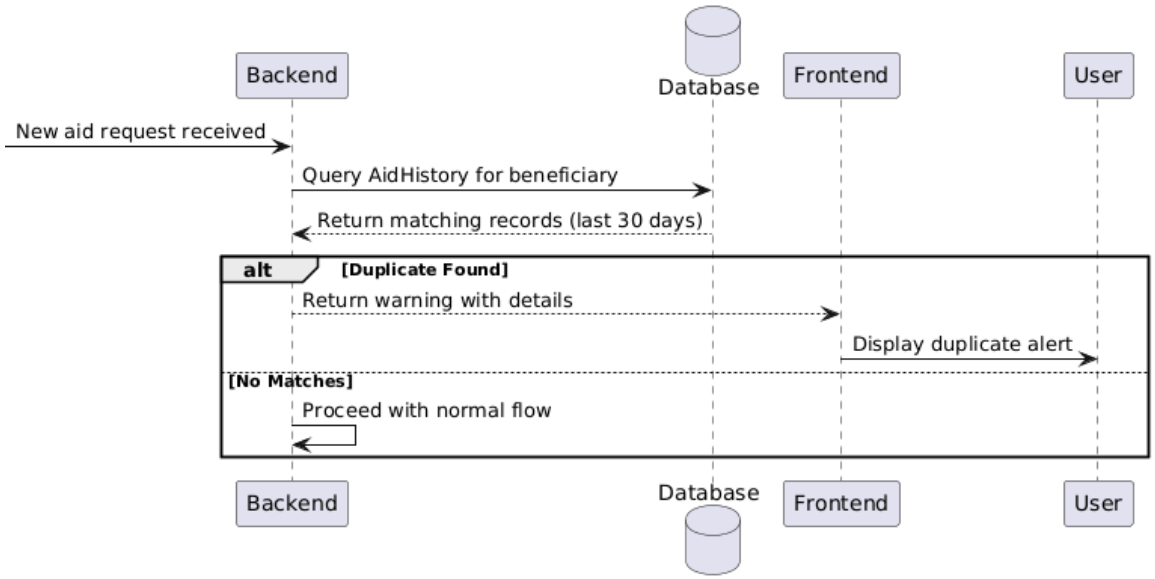
User Login Workflow (Sequence 1)



Create Aid Request Workflow (Sequence 2)



Record Delivery Workflow (Sequence 5)



Duplication Check Logic (Sequence 6)